

Stringhe avanzate

Stringhe avanzate

Metodi avanzati per la manipolazione delle stringhe in Python.

Qual è la funzione delle stringhe avanzate

Le stringhe sono ovunque: nomi, messaggi, dati da file, risposte dal web. Python ha decine di strumenti pronti per cercare, modificare, tagliare e unire testo. In questa pagina vediamo quelli più utili nella pratica.

Cercare testo dentro una stringa

`find()` — trova la posizione di una sottostringa

```
testo = "Ciao, mi chiamo Alice e vivo a Roma."

print(testo.find("Alice"))    # 15 → "Alice" inizia alla posizione 15
print(testo.find("Mario"))    # -1 → non trovato (restituisce -1 invece di
                               # errore)
print(testo.rfind("a"))       # 31 → posizione dell'ULTIMA "a"
```

`count()` — conta quante volte appare qualcosa

```
testo = "banana"
print(testo.count("a"))      # 3
print(testo.count("na"))     # 2
```

`replace()` — sostituisci una parte del testo

```
testo = "Ciao mondo! Ciao Python!"
print(testo.replace("Ciao", "Salve"))      # Salve mondo! Salve Python!
print(testo.replace("Ciao", "Salve", 1))   # Salve mondo! Ciao Python!
(solo la prima)
```

Tagliare e unire

`split()` — divide una stringa in una lista di pezzi

```
frase = "mela banana ciliegia uva"
parole = frase.split()                  # Divide sugli spazi (di default)
print(parole)    # ['mela', 'banana', 'ciliegia', 'uva']

# Dividere su un altro separatore
dati = "Alice,16,Roma"
campi = dati.split(",")                 # Divide sulle virgole
print(campi)    # ['Alice', '16', 'Roma']

# Limitare il numero di divisioni
print("a:b:c:d".split(":", 2))          # ['a', 'b', 'c:d'] ← si ferma dopo 2
tagli
```

`splitlines()` — divide su ogni fine riga

```
testo = "Prima riga\nSeconda riga\nTerza riga"
righe = testo.splitlines()
print(righe)    # ['Prima riga', 'Seconda riga', 'Terza riga']
```

`join()` — unisce una lista di stringhe in una sola

`join()` è l'opposto di `split()` : prende una lista e la incolla insieme usando un separatore.

```
parole = ["mela", "banana", "ciliegia"]
print(", ".join(parole))    # mela, banana, ciliegia
print(" - ".join(parole))   # mela - banana - ciliegia
print("".join(parole))      # melebananaciliegia
```

Pulizia del testo

Rimuovere spazi in eccesso

```
testo = "    ciao mondo    "
print(testo.strip())      # "ciao mondo"      ← rimuove da entrambi i lati
print(testo.lstrip())     # "ciao mondo    "  ← rimuove solo a sinistra
print(testo.rstrip())     # "    ciao mondo"  ← rimuove solo a destra

# Rimuovere caratteri specifici (non solo spazi)
testo2 = "###ciao###"
print(testo2.strip("#"))  # "ciao"
```

Cambiare maiuscole e minuscole

```
testo = "ciao mondo"
print(testo.upper())      # CIAO MONDO
print(testo.lower())      # ciao mondo
print(testo.capitalize()) # Ciao mondo ← solo la prima lettera
                           maiuscola
print(testo.title())      # Ciao Mondo ← prima lettera di ogni parola
                           maiuscola
print(testo.swapcase())   # CIAO MONDO ← inverte: minuscole↔maiuscole
```

Verificare il contenuto di una stringa

Questi metodi restituiscono `True` o `False` :

```
print("hello".islower())    # True ← tutto minuscolo
print("HELLO".isupper())    # True ← tutto maiuscolo
print("Hello World".istitle()) # True ← ogni parola inizia con maiuscola

print("abc123".isalnum())   # True ← solo lettere e numeri
print("abc".isalpha())      # True ← solo lettere
print("123".isdigit())      # True ← solo cifre
print(" ".isspace())        # True ← solo spazi
```

Allineamento e riempimento

Utili per creare tabelle o output ben formattato:

```
print("42".zfill(5))          # "00042"      ← riempie con zeri a
sinistra
print("ciao".ljust(10))       # "ciao      " ← allinea a sinistra (spazi
a destra)
print("ciao".rjust(10))       # "      ciao" ← allinea a destra (spazi a
sinistra)
print("ciao".center(10))      # "   ciao   " ← centra
print("ciao".center(10, "-")) # "---ciao---" ← centra con carattere
personalizzato
```

Dividere in tre parti con `partition()`

`partition()` divide la stringa in esattamente tre parti: prima del separatore, il separatore stesso, dopo il separatore.

```
testo = "nome: Alice"
prima, sep, dopo = testo.partition(":")
print(prima)    # "nome"
print(sep)      # ":"
print(dopo)     # " Alice"
```

Esempio pratico: leggere dati separati da virgole

Un formato molto comune per i dati è il CSV (valori separati da virgole). Ecco come estrarre le informazioni da una riga:

```
riga_csv = "Alice,16,Roma,8.5"
campi = riga_csv.split(",") # Divide sulle virgole

nome = campi[0].strip()      # Rimuove eventuali spazi
eta = int(campi[1].strip())  # Converte in numero intero
citta = campi[2].strip()
voto = float(campi[3].strip()) # Converte in numero decimale

print(f"{nome} ha {eta} anni, vive a {citta} e ha voto {voto}")
# → Alice ha 16 anni, vive a Roma e ha voto 8.5
```